

LÊN MÁY BAY**Giới hạn thời gian : 1 giây****Giới hạn bộ nhớ: 512 Mb**

Trên máy bay của hãng hàng không Bitavia, ghế được sắp xếp thành n hàng, mỗi hàng có sáu chỗ ngồi và có một lối đi giữa chỗ ngồi thứ ba và thứ tư. Một số hành khách đăng ký trước trực tuyến, trong khi những hành khách khác đăng ký tại quầy đăng ký ở sân bay. Khi đăng ký trước trực tuyến, hành khách có thể chọn bất kỳ chỗ nào và không thể thay đổi sau đó. Ví dụ, với $n = 6$, sắp xếp chỗ ngồi trong máy bay sau khi đăng ký trước trực tuyến có thể trông như sau (chỗ ngồi đã được đánh dấu bằng dấu X):

	1	2	3	4	5	6
1	X					
2						
3					X	
4	X					
5						
6			X	X		

Khi có m hành khách đến quầy đăng ký, theo quy tắc của Bitavia, họ cần được sắp xếp vào máy bay sao cho sắp xếp cuối cùng trong máy bay là đối xứng qua lối đi.

Điều này có nghĩa là nếu có hành khách ngồi trên ghế đầu tiên của một hàng, thì cũng phải có hành khách ngồi trên ghế thứ sáu của hàng đó. Tương tự, điều đó cũng đúng cho ghế thứ hai và thứ năm, ghế thứ ba và thứ tư tương ứng. Đồng thời, không được phép chuyển chỗ cho những hành khách đã đăng ký trước trực tuyến. Bằng cách này, ta có thể thêm vào sắp xếp ban đầu như đã được chỉ ra trên hình minh họa, với bảy hành khách, tuân theo điều kiện đối xứng.

	1	2	3	4	5	6
1	☒	☐	☐	☐	☐	☒
2	☒	☐	☐	☐	☐	☒
3	☐	☒	☐	☐	☒	☐
4	☒	☐	☐	☐	☐	☒
5	☐	☐	☒	☒	☐	☐
6	☐	☐	☒	☒	☐	☐

Bạn đã được cung cấp sơ đồ ghế cho hành khách sau khi đăng ký trực tuyến. Bạn cần phải sắp xếp m hành khách sao cho sơ đồ ghế cuối cùng trên máy bay là đối xứng qua lối đi hoặc xác định rằng điều này là không thể.

Định dạng dữ liệu vào từ tập tin văn bản plane.inp :

Dòng đầu tiên chứa hai số nguyên n và m - số hàng trên máy bay và số hành khách đến quầy đăng ký ($1 \leq n \leq 1000$, $0 \leq m \leq 6000$).

Trong n dòng tiếp theo, sơ đồ ghế ban đầu trên máy bay sau khi đăng ký trực tuyến được mô tả. Mỗi dòng chứa sáu ký tự, trong đó ký tự thứ i của dòng thứ j là "X" (in hoa) nếu ghế thứ i trong hàng j đã được chiếm và "." (dấu chấm) nếu chưa.

Định dạng dữ liệu ra tập tin văn bản plane.out:

Nếu không tồn tại sơ đồ ghế như mong muốn, in ra "Impossible".

Nếu có, in ra n dòng, mỗi dòng chứa sáu ký tự, biểu diễn sơ đồ ghế cuối cùng trên máy bay. Mỗi ký tự thứ i của dòng thứ j phải là "X" nếu ghế thứ i trong hàng j đã được chiếm, và "." nếu chưa. Nếu có nhiều giải pháp, bạn có thể in ra một giải pháp bất kỳ.

Ví dụ:

plane.inp	plane.out
1 0 X.XX.X	X.XX.X
2 1 X.XX.X ..X..	X.XX.X ..XX..
3 2 X.XX.X X..X.X	Impossible
1 103 .X.XXX	Impossible
6 7 X.....X. X.....XX..	X....X X....X .X..X. X....X ..XX.. ..XX..

Hệ thống bảng đánh giá

Điểm cho mỗi bài toán con sẽ được tính toán chỉ khi tất cả các bài kiểm tra cho bài toán con và các bài toán con cần thiết đã được vượt qua thành công.

Bài toán con	Số điểm	Thêm giới hạn	Bài toán con cần thiết	Thông tin về kiểm tra
1	15	$m = 1$		Lỗi đầu tiên
2	16	ban đầu tất cả các ghế trên máy bay đều trống		Lỗi đầu tiên
3	17	$m = 1$		Lỗi đầu tiên
4	18	chỉ có một ghế bị chiếm ban đầu		Lỗi đầu tiên
5	34	Không	1–4	Lỗi đầu tiên

Giải thích ví dụ:

Dưới đây là năm ví dụ về dữ liệu đầu vào:

- 1) Trong ví dụ đầu tiên, $m = 0$, và sự sắp xếp trên máy bay là đối xứng, vì vậy sự sắp xếp cuối cùng trùng với sự sắp xếp ban đầu.
- 2) Trong ví dụ thứ hai, chỉ có một cách để sắp xếp hành khách một cách đối xứng.
- 3) Trong ví dụ thứ ba, có một giải pháp khi $m = 1$, nhưng khi $m = 2$, không tồn tại cách nào để sắp xếp tất cả hành khách một cách đối xứng.
- 4) Trong ví dụ thứ tư, cần sắp xếp nhiều hành khách hơn số chỗ trống trên máy bay.
- 5) Ví dụ thứ năm tương ứng với tình huống được mô tả trong hình minh họa của văn bản điều kiện. Trong ví dụ này, có nhiều giải pháp, và một trong số đó được trình bày.

DÃY SỐ BITONIC

Giới hạn thời gian : 1 giây

Giới hạn bộ nhớ: 512 Mb

Một dãy số $[b_1, b_2, \dots, b_k]$ được gọi là **bitonic** nếu thỏa mãn bất đẳng thức: $b_1 < b_2 < \dots < b_i > \dots > b_k$ với một giá trị $1 \leq i \leq k$.

Ví dụ:

- Các dãy số: $[1]$, $[1, 2, 3, 2]$, $[1, 4, 10]$, $[3, 2]$ là các dãy bitonic.
- Các dãy số: $[1, 1]$, $[2, 1, 3]$ không phải là bitonic.

Yêu cầu:

Cho một dãy số $[a_1, a_2, \dots, a_n]$. Hãy đếm số cặp chỉ số (l, r) sao cho: $1 \leq l \leq r \leq n$ và dãy con $[a_l, a_{l+1}, \dots, a_r]$ là một dãy bitonic.

Định dạng dữ liệu vào từ tập tin văn bản bitonic.inp :

Dòng đầu tiên chứa số nguyên n ($1 \leq n \leq 300000$).

Dòng thứ hai chứa n số nguyên $a_1, a_2, \dots, a_n$ ($1 \leq a_i \leq n$).

Định dạng dữ liệu ra tập tin văn bản bitonic.out:

In ra một số nguyên — số lượng cặp (l, r) sao cho $1 \leq l \leq r \leq n$ và dãy con $[a_l, a_{l+1}, \dots, a_r]$ là **bitonic**.

Ví dụ:

bitonic.inp	bitonic.out
5 1 1 2 3 1	11
3 1 1 1	3

Hệ thống bảng đánh giá

Điểm cho mỗi bài toán con sẽ được tính toán chỉ khi tất cả các bài kiểm tra cho bài toán con và các bài toán con cần thiết đã được vượt qua thành công.

Bài toán con	Số điểm	Thêm giới hạn	Bài toán con cần thiết	Thông tin về kiểm tra
1	27	$n \leq 500$		Lỗi đầu tiên
2	14	$n \leq 500$	1	Lỗi đầu tiên
3	20	Tất cả các số a_i khác nhau		Lỗi đầu tiên
4	39	Không	1–3	Lỗi đầu tiên

Giải thích ví dụ:

Trong ví dụ đầu tiên, các cặp phù hợp là:

- (1, 1), dãy [1]
- (2, 2), dãy [1]
- (2, 3), dãy [1, 2]
- (2, 4), dãy [1, 2, 3]
- (2, 5), dãy [1, 2, 3, 1]
- (3, 3), dãy [2]
- (3, 4), dãy [2, 3]
- (3, 5), dãy [2, 3, 1]
- (4, 4), dãy [3]
- (4, 5), dãy [3, 1]
- (5, 5), dãy [1]

TRÒ CHƠI VỚI BẢNG

Giới hạn thời gian : 1 giây

Giới hạn bộ nhớ: 512 Mb

Bạn được cung cấp một bảng (ma trận) A kích thước $h \times w$, trong đó mỗi ô chứa một số nguyên. Bạn được phép thực hiện hai loại thao tác:

- **Xóa một cột bất kỳ:** Sau khi xóa, các cột bên trái và bên phải của cột bị xóa sẽ trở thành cột liền kề.
- **Xóa một hàng bất kỳ:** Sau khi xóa, các hàng bên trên và bên dưới của hàng bị xóa sẽ trở thành hàng liền kề.

Mục tiêu là xác định xem có thể thực hiện các thao tác này để làm cho tổng các số trong bảng bằng một giá trị cho trước s. Nếu có thể, cần chỉ rõ trình tự thực hiện các thao tác (hàng/cột nào bị xóa).

Định dạng dữ liệu vào từ tập tin văn bản gametable.inp :

Dòng đầu tiên: Chứa hai số nguyên h và w ($1 \leq h, w \leq 15$), là chiều cao và chiều rộng của bảng.

h dòng tiếp theo: Mỗi dòng chứa w số nguyên $A[i][j]$ ($0 \leq A[i][j] \leq 10^9$), là các giá trị trong bảng.

Dòng cuối cùng: Chứa một số nguyên s ($1 \leq s \leq 10^{18}$), là tổng cần đạt được.

Định dạng dữ liệu ra tập tin văn bản gametable.out:

Nếu không thể biến đổi bảng gốc thành bảng có tổng các số bằng s, hãy in ra dòng chữ "NO".

Ngược lại:

- Trong dòng đầu tiên, in ra dòng chữ "YES".
- Trong dòng thứ hai, in ra số nguyên k — số lượng thao tác cần thực hiện trên bảng để biến đổi thành bảng có tổng các số bằng s.
- Trong mỗi k dòng tiếp theo, in ra hai số nguyên t_j và i_j :

- $t_j=1$: nếu thao tác được thực hiện trên một hàng.
- $t_j=2$: nếu thao tác được thực hiện trên một cột.
- i_j là số thứ tự của hàng hoặc cột trong bảng gốc mà thao tác được thực hiện.

Ví dụ:

gametable.inp	gametable.out
3 3 1 2 3 2 3 1 3 1 2 8	YES 2 1 3 2 3
2 3 2 2 2 2 2 2 5	NO
5 5 1 2 1 4 5 2 5 4 1 2 4 2 4 3 1 5 5 3 2 4 1 2 4 5 2 34	YES 3 1 4 1 5 2 1

Hệ thống bảng đánh giá

Điểm cho mỗi bài toán con sẽ được tính toán chỉ khi tất cả các bài kiểm tra cho bài toán con và các bài toán con cần thiết đã được vượt qua thành công.

Bài toán con	Số điểm	Thêm giới hạn	Bài toán con cần thiết	Thông tin về kiểm tra
1	17	$h = 1$		Lỗi đầu tiên

2	6	Tổng số trên dòng i không vượt quá i		Lỗi đầu tiên
3	10	$h \leq 3$	1	Lỗi đầu tiên
4	13	$h, w \leq 10$		Lỗi đầu tiên
5	13	$h, w \leq 12$	4	Lỗi đầu tiên
6	12	$a_{i,j} \leq 6$		Lỗi đầu tiên
7	29		1-6	Lỗi đầu tiên

Giải thích ví dụ:

Trong ví dụ đầu tiên đưa ra bảng sau:

1	2	3
2	3	1
3	1	2

Xoá dòng thứ ba và và cột thứ ba ta nhận được bảng có tổng bằng 8

1	2	3
2	3	1
3	1	2

 $\rightarrow$

1	2	3
2	3	1

 $\rightarrow$

1	2
2	3

Trong ví dụ 2 có thể chỉ ra các lệnh cho phép không thể nhận được bảng có tổng bằng 5 từ bảng ban đầu.

Ở ví dụ 3 bảng ban đầu như sau:

1	2	1	4	5
2	5	4	1	2
4	2	4	3	1
5	5	3	2	4
1	2	4	5	2

Xoá hai dòng cuối và cột đầu tiên ta nhận được bảng có tổng bằng 34:

1	2	1	4	5
2	5	4	1	2
4	2	4	3	1
5	5	3	2	4
1	2	4	5	2

→

1	2	1	4	5
2	5	4	1	2
4	2	4	3	1
5	5	3	2	4

→

1	2	1	4	5
2	5	4	1	2
4	2	4	3	1

→

2	1	4	5
5	4	1	2
2	4	3	1